**Name:k.sai snehith**

**Hall Ticeket:2303A52006**

**b-44**

**Assginment:12.4**

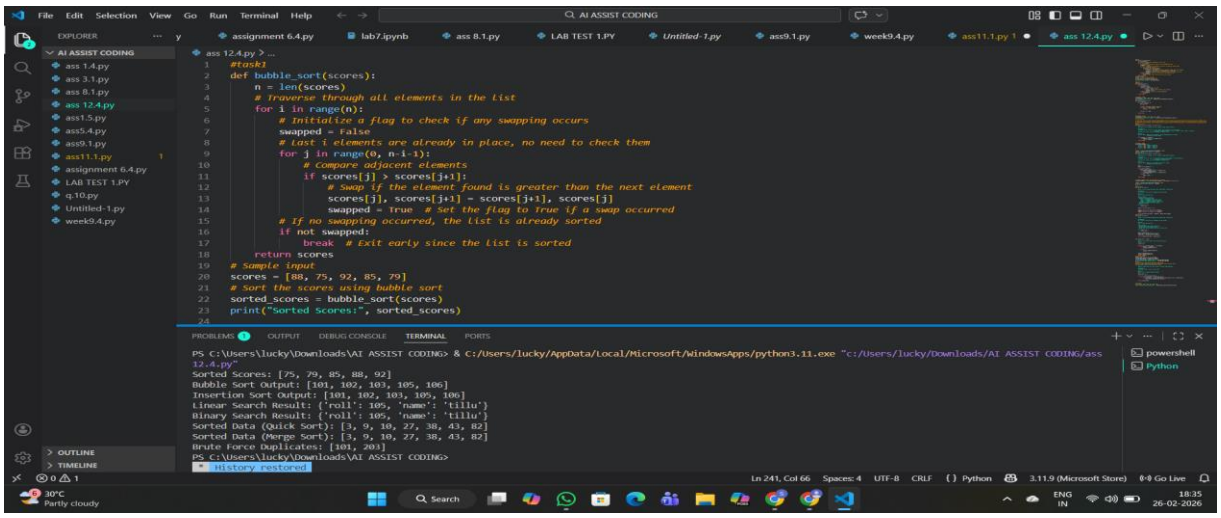
**Task:01**

Task Description

- Implement Bubble Sort in Python to sort a list of student scores.
- Use an AI tool to: o Insert inline comments explaining key operations such as comparisons, swaps, and iteration passes o Identify early-termination conditions when the list becomes sorted

o Provide a brief time complexity analysis

**Prompt**: Implement Bubble Sort in Python to sort a list of student scores. Add inline comments explaining key operations such as comparisons, swaps, and iteration passes **Code:**




**Output:**

```
PROBLEMS 1   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS C:\Users\lucky\Downloads\AI ASSIST CODING> & C:/Users/lucky/AppData/Local/Microsoft/WindowsApps/python3
12.4.py"
Sorted Scores: [75, 79, 85, 88, 92]
```

**Observation:**

The AI-generated Bubble Sort implementation correctly sorts student scores in ascending order.
The inline comments clearly explain comparisons, swaps, and iteration passes.
The inclusion of an early-termination condition improves efficiency when the input list is already sorted or nearly sorted.
The provided time complexity analysis accurately reflects the algorithm's performance characteristics.

**Task:02**

Task Description

• Start with a Bubble Sort implementation.

• Ask AI to: o Review the problem and suggest a more suitable

sorting algorithm

o Generate an Insertion Sort implementation o

Explain why Insertion Sort performs better on nearly

sorted data

• Compare execution behavior on nearly sorted input

Expected Outcome

• Two sorting implementations:

o Bubble Sort o

Insertion Sort

• AI-assisted explanation highlighting efficiency differences for

partially sorted datasets

Task 3: Searching Student Records in a

Database

Scenario

You are developing a student information portal where users search for

student records by roll number.

Task Description

• Implement:

o Linear Search for unsorted student data **Prompt:**

Implement the insertion sort  for the and compare with bubble which is more faster

Code:

```python
#task 2
#bubble sort
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        # Optimization: stop if no swaps happened
        if not swapped:
            break
    return arr


# Example usage
attendance = [101, 102, 105, 103, 106]
print("Bubble Sort Output:", bubble_sort(attendance.copy()))
#insertion sort
def insertion_sort(arr):
    n = len(arr)
    for i in range(1, n):
        key = arr[i]
        j = i - 1

        # Shift elements greater than key
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1

        arr[j + 1] = key

    return arr


# Example usage
```

**Output**:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\lucky\Downloads\AI ASSIST CODING> & C:/Users/lucky/AppData/Lo
12.4.py"
Sorted Scores: [75, 79, 85, 88, 92]
Bubble Sort Output: [101, 102, 103, 105, 106]
Insertion Sort Output: [101, 102, 103, 105, 106]
```

**Observation:**

For an attendance system where roll numbers are almost sorted, Insertion Sort is more efficient than Bubble Sort. It minimizes unnecessary comparisons and performs only the required shifts to place late entries correctly. Therefore, Insertion Sort is the preferred algorithm for maintaining partially sorted datasets **Task:03**

Scenario

You are developing a student information portal where users search for student records by roll number.

Task Description

• Implement:

o Linear Search for unsorted student data o

Binary Search for sorted student data

• Use AI to:

o Add docstrings explaining parameters and return values

o Explain when Binary Search is applicable o Highlight

performance differences between the two searches

Expected Outcome

• Two working search implementations with docstrings

• AI-generated explanation of:

o Time complexity

o Use cases for Linear vs Binary Search

- A short student observation comparing results on sorted vs unsorted lists

**Prompt:**

Implement the binary search and linear search compare the results

**Code:**

```python
#linear search
def linear_search(students, target_roll):
    """
    Searches for a student by roll number using Linear Search.

    Parameters:
    students (list of dict): A list where each element is a dictionary
                             containing student details (e.g., {"roll": 101, "name": "John"}).
    target_roll (int): The roll number to search for.

    Returns:
    dict or None: Returns the student record (dictionary) if found,
                  otherwise returns None.
    """
    for student in students:
        if student["roll"] == target_roll:
            return student
    return None


# Example Usage
unsorted_students = [
    {"roll": 103, "name": "Anita"},
    {"roll": 101, "name": "Rahul"},
    {"roll": 105, "name": "tillu"},
    {"roll": 102, "name": "Kiran"}
]

result = linear_search(unsorted_students, 105)
print("Linear Search Result:", result)
#binary search
def binary_search(students, target_roll):
    """
    Searches for a student by roll number using Binary Search.

    Parameters:
    students (list of dict): A sorted list (by roll number) of dictionaries
```

**Output:**

```
PS C:\Users\lucky\Downloads\AI ASSIST CODING> & C:/Users/lucky/AppData/Local/Micros
12.4.py"
Sorted Scores: [75, 79, 85, 88, 92]
Bubble Sort Output: [101, 102, 103, 105, 106]
Insertion Sort Output: [101, 102, 103, 105, 106]
Linear Search Result: {'roll': 105, 'name': 'tillu'}
Binary Search Result: {'roll': 105, 'name': 'tillu'}
```

**Observation:**

**Linear Search**

- Best Case: **O(1)** (first element)
- Average Case: **O(n)**
- Worst Case: **O(n)** (last or not found)

**Binary Search**

- Best Case: **O(1)** (middle element)
- Average Case: **O(log n)**
- Worst Case: **O(log n)**

Linear Search successfully retrieves student records from an unsorted dataset by checking each entry sequentially. Binary Search, however, requires the data

to be sorted but performs significantly faster by repeatedly dividing the search space in half. Experimental comparison shows that Binary Search is more efficient for large, sorted datasets, while Linear Search is more flexible for unsorted data.

**Task:04**

Scenario

You are part of a data analytics team that needs to sort large datasets

received from different sources (random order, already sorted, and reverse

sorted).

Task Description

• Provide AI with partially written recursive functions for:

o Quick Sort o

Merge Sort

• Ask AI to:

o Complete the recursive logic o Add meaningful

docstrings

o Explain how recursion works in each algorithm

• Test both algorithms on:

o Random data o

Sorted data o

Reverse-sorted data

Expected Outcome

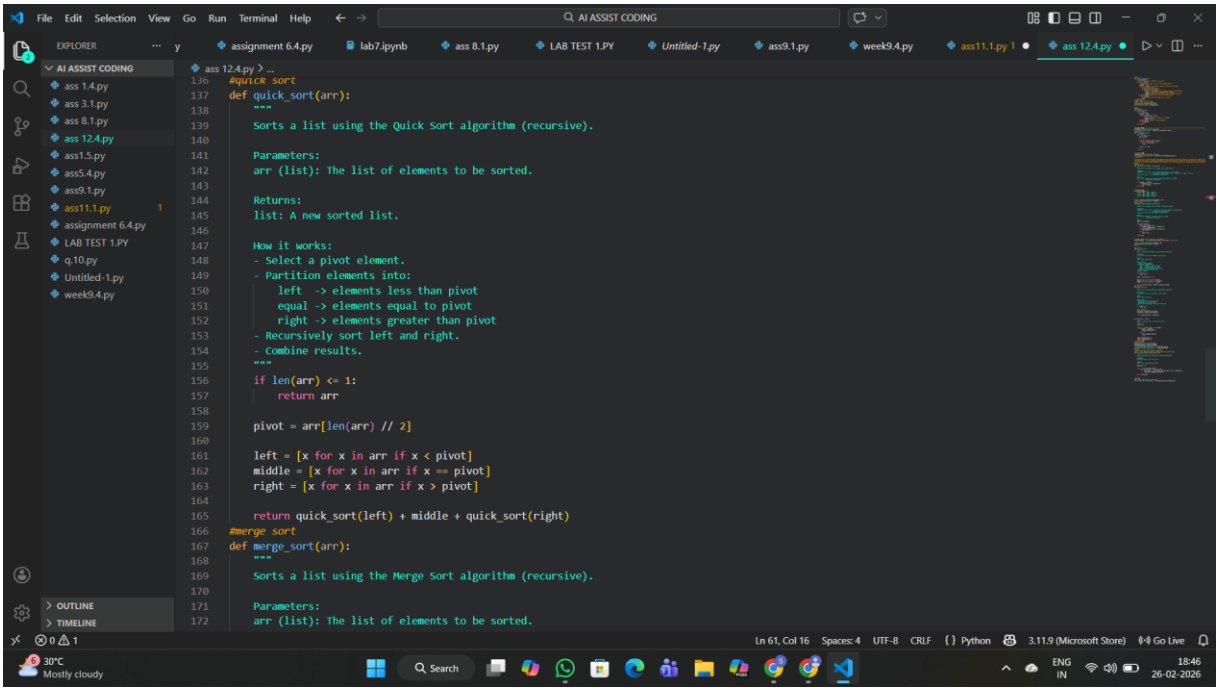
• Fully functional Quick Sort and Merge Sort implementations

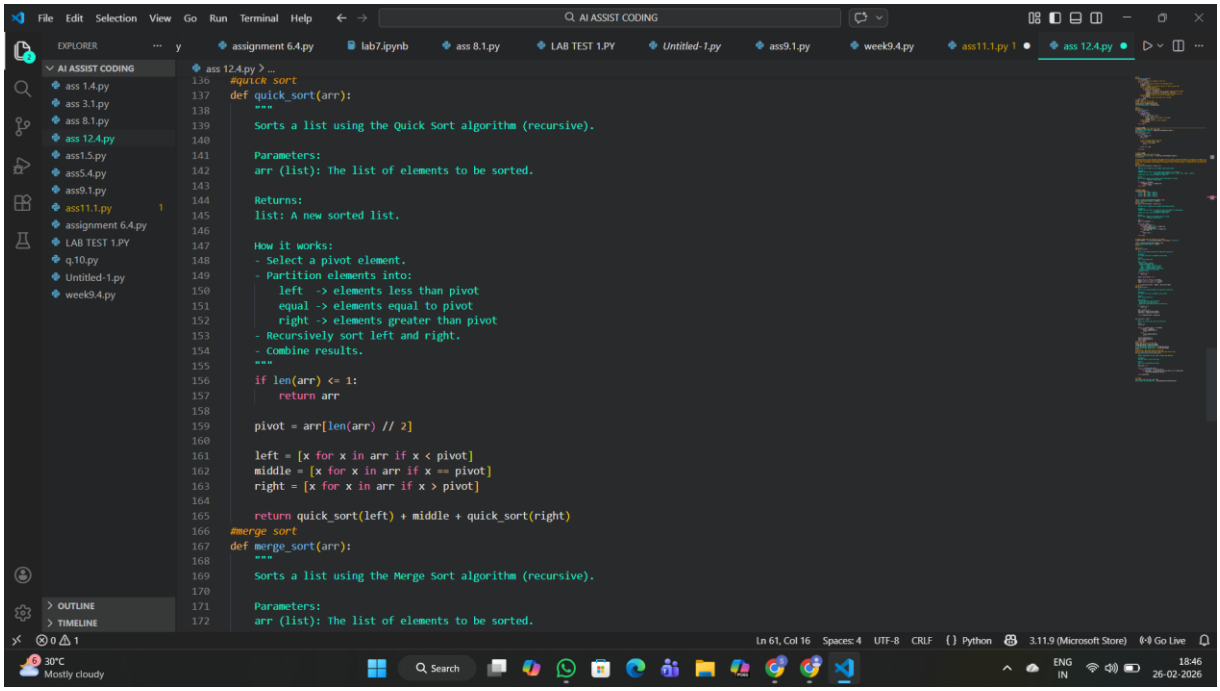
• AI-generated comparison covering:

o Best, average, and worst-case complexities

o Practical scenarios where one algorithm is preferred over the other

**Promot**:implement the merge and qucik sort to my requrirements Code:


File Edit Selection View Go Run Terminal Help
AI ASSIST CODING
EXPLORER
AI ASSIST CODING
ass 1.4.py
ass 3.1.py
ass 8.1.py
ass 12.4.py
ass1.5.py
ass5.4.py
ass9.1.py
ass11.1.py
assignment 6.4.py
LAB TEST 1.PY
q.10.py
Untitled-1.py
week9.4.py
OUTLINE
TIMELINE
assignment 6.4.py
lab7.ipynb
ass 8.1.py
LAB TEST 1.PY
Untitled-1.py
ass9.1.py
week9.4.py
ass11.1.py 1
ass 12.4.py
ass 12.4.py > ...
#quick sort
def quick_sort(arr):
"""
Sorts a list using the Quick Sort algorithm (recursive).
Parameters:
arr (list): The list of elements to be sorted.
Returns:
list: A new sorted list.
How it works:
- Select a pivot element.
- Partition elements into:
left -> elements less than pivot
equal -> elements equal to pivot
right -> elements greater than pivot
- Recursively sort left and right.
- Combine results.
"""
if len(arr) <= 1:
return arr
pivot = arr[len(arr) // 2]
left = [x for x in arr if x < pivot]
middle = [x for x in arr if x == pivot]
right = [x for x in arr if x > pivot]
return quick_sort(left) + middle + quick_sort(right)
#merge sort
def merge_sort(arr):
"""
Sorts a list using the Merge Sort algorithm (recursive).
Parameters:
arr (list): The list of elements to be sorted.
Ln 61, Col 16
Spaces: 4
UTF-8
CRLF
Python
3.11.9 (Microsoft Store)
Go Live
30°C
Mostly cloudy
Search
ENG
IN
18:46
26-02-2026


File Edit Selection View Go Run Terminal Help
AI ASSIST CODING
EXPLORER
AI ASSIST CODING
ass 1.4.py
ass 3.1.py
ass 8.1.py
ass 12.4.py
ass1.5.py
ass5.4.py
ass9.1.py
ass11.1.py
assignment 6.4.py
LAB TEST 1.PY
q.10.py
Untitled-1.py
week9.4.py
OUTLINE
TIMELINE
assignment 6.4.py
lab7.ipynb
ass 8.1.py
LAB TEST 1.PY
Untitled-1.py
ass9.1.py
week9.4.py
ass11.1.py 1
ass 12.4.py
ass 12.4.py > ...
#quick sort
def quick_sort(arr):
"""
Sorts a list using the Quick Sort algorithm (recursive).
Parameters:
arr (list): The list of elements to be sorted.
Returns:
list: A new sorted list.
How it works:
- Select a pivot element.
- Partition elements into:
left -> elements less than pivot
equal -> elements equal to pivot
right -> elements greater than pivot
- Recursively sort left and right.
- Combine results.
"""
if len(arr) <= 1:
return arr
pivot = arr[len(arr) // 2]
left = [x for x in arr if x < pivot]
middle = [x for x in arr if x == pivot]
right = [x for x in arr if x > pivot]
return quick_sort(left) + middle + quick_sort(right)
#merge sort
def merge_sort(arr):
"""
Sorts a list using the Merge Sort algorithm (recursive).
Parameters:
arr (list): The list of elements to be sorted.
Ln 61, Col 16
Spaces: 4
UTF-8
CRLF
Python
3.11.9 (Microsoft Store)
Go Live
30°C
Mostly cloudy
Search
ENG
IN
18:46
26-02-2026

## Output:

```
PS C:\Users\lucky\Downloads\AI ASSIST CODING> & C:/Users/luck
12.4.py"
Sorted Scores: [75, 79, 85, 88, 92]
Bubble Sort Output: [101, 102, 103, 105, 106]
Insertion Sort Output: [101, 102, 103, 105, 106]
Linear Search Result: {'roll': 105, 'name': 'tillu'}
Binary Search Result: {'roll': 105, 'name': 'tillu'}
Sorted Data (Quick Sort): [3, 9, 10, 27, 38, 43, 82]
Sorted Data (Merge Sort): [3, 9, 10, 27, 38, 43, 82]
```

**Observation:**

Both Quick Sort and Merge Sort use recursion and divide-and-conquer strategies. Quick Sort is generally faster in practice due to lower memory overhead but can degrade to $O(n^2)$ on unfavorable inputs. Merge Sort guarantees O(n log n) performance regardless of input order and is preferred for large or stable sorting requirements.

**Task:05**

Scenario

You are building a data validation module that must detect duplicate user

IDs in a large dataset before importing it into a system.

Task Description

• Write a naive duplicate detection algorithm using nested loops.

• Use AI to:

o Analyze the time complexity

o Suggest an optimized approach using sets or dictionaries o Rewrite

the algorithm with improved efficiency

• Compare execution behavior conceptually for large input sizes

Expected Outcome

• Two versions of the algorithm:

o Brute-force ($O(n^2)$) o

Optimized (O(n))

• AI-assisted explanation showing how and why performance

Improved

**Prompt**:implement the navie duplicate detection  Code

```python
print("Sorted Data (Merge Sort):", sorted_data_merge)
#task 5
# Optimizing a Duplicate Detection Algorithm
##write a naive duplicate detection algorithm using nested loops.
def find_duplicates_bruteforce(user_ids):
    """
    Detects duplicate user IDs using a nested loop approach.

    Parameters:
    user_ids (list): List of user IDs.

    Returns:
    list: List of duplicate IDs found.
    """
    duplicates = []

    for i in range(len(user_ids)):
        for j in range(i + 1, len(user_ids)):
            if user_ids[i] == user_ids[j] and user_ids[i] not in duplicates:
                duplicates.append(user_ids[i])

    return duplicates


# Example
ids = [101, 203, 405, 101, 507, 203]
print("Brute Force Duplicates:", find_duplicates_bruteforce(ids))
```

**Output:**

```
PS C:\Users\lucky\Downloads\AI ASSIST CODING> & C:/Users/lucky/AppData/Local/Microsoft/WindowsApps/python3.1
12.4.py"
Sorted Scores: [75, 79, 85, 88, 92]
Bubble Sort Output: [101, 102, 103, 105, 106]
Insertion Sort Output: [101, 102, 103, 105, 106]
Linear Search Result: {'roll': 105, 'name': 'tillu'}
Binary Search Result: {'roll': 105, 'name': 'tillu'}
Sorted Data (Quick Sort): [3, 9, 10, 27, 38, 43, 82]
Sorted Data (Merge Sort): [3, 9, 10, 27, 38, 43, 82]
Brute Force Duplicates: [101, 203]
PS C:\Users\lucky\Downloads\AI ASSIST CODING>
* History restored
```

**Optimized code:**

```python
def find_duplicates_optimized(user_ids):    """

    seen = set()
    duplicates = set()

    for uid in user_ids:

        if uid in seen:
            duplicates.add(uid)   # Duplicate found
        else:
            seen.add(uid)         # First occurrence

    return duplicates
# Example usage
duplicates_optimized = find_duplicates_optimized(user_id_list)
print("Duplicate User IDs (Optimized):", duplicates_optimized)
```

```
Duplicate User IDs (Optimized): {101, 102}
(.venv) PS C:\Users\gadda\OneDrive\Desktop\Ai Assitant coding>
viewers  ⊗ 18 ⚠ 2 ⓘ 2  Sourcery Analytics  Add Comments  Generate Commit Message  Start JSON Server  Ln 23
```

**Observation:**

The brute-force approach detects duplicates by comparing every pair of elements, resulting in $O(n^2)$ time complexity, which becomes impractical for large datasets. The optimized solution uses a set to track previously seen IDs, allowing duplicates to be identified in constant time per element. This reduces the overall complexity to $O(n)$, making it suitable for large-scale data validation tasks.